

Reflection Template

Use Case	UC-11
Use Case Name	Unassign Task
Requirement IDs	FR12, FR20, FR22, FR23
Matching Feature Comparison Sets	CS-23
Implementation Order	Java first -> Kotlin second

Feature Relevance

Feature/s:

Null safety, extension functions

Did the feature/s occur naturally in this/these context/s?

Yes, setting assignedUser to null is the most direct null safety moment. Kotlin shows this naturally cause assignedUser is typed as User?. Extension function was reused as usual.

Qualitative Ratings (1-5)

- 1 = very poor/very difficult
- 2 = rather poor/difficult
- 3 = neutral/moderate
- 4 = good/easy
- 5 = very good/very easy

Criteria	Java	Kotlin
Readability	4	5
Compactness of logic	5	5
Perceived implementation effort	5	5

Justifications

Readability:

task.assignedUser = null in Kotlin reads more naturally than task.setAssignedUser(null) in Java. Otherwise both identical.

Compactness of logic:

Both are equally compact. This is the simplest use case so far. No meaningful difference.

Implementation effort:

Both straightforward and easy. Nothing new.

Observed structural differences

Setting a nullable field to null:

Kotlin: `task.assignedUser = null`

Java: `task.setAssignedUser(null)`

Same setter vs direct assignment pattern as UC8 and UC10.

No new structural differences compared to previous use cases.

Final comparative summary

UC11 is the simplest use case so far and confirms existing patterns. The only feature-relevant observation is null safety `assignedUser` being typed as `User?` in Kotlin makes the intent of setting it to null immediately clear from the type alone.